Stone Storm: owner's release and management guide

This guide is for you (the owner) or a simple AI helper. It explains how to build the game, send it to Google Play, test it safely, go public, ship updates and look after the game afterwards. Follow the steps in order. Every command is copied exactly; you only change the parts written in CAPITALS.

Already done (you don't need to redo these): Google Play developer account, payments profile with BillDesk, upload key, AdMob account + 2 ad units + consent messages, Firebase, internal testing track with testers, license testers, the 11 in-app products, your phone as an AdMob test device, website + privacy policy + app-ads.txt.

Still to do before the public release: part 3 (one-time: let the tool upload), part 7 (closed test with 12 testers for 14 days, then the forms and the store listing), part 8 (production).

Contents

1. Words you will see
2. Your secret files (back them up today)
3. One-time setup: let the tool upload to Google Play
4. Test mode and real mode: ads, purchases, testers
5. Making a new version (the everyday steps)
6. Play tracks: internal, closed, open, production
7. Before the first public release (checklist)
8. Going public: production and staged rollout
9. Changing things later: what needs a new version
10. How players get updates
11. After release: what to check and when
12. Money: Play earnings, AdMob earnings, refunds, taxes
13. Your phone, friends' phones and saved progress
14. Setting up a new Mac
15. Troubleshooting
16. Command cheat sheet
17. Working with an AI helper

---

1. Words you will see

Word	Meaning
AAB (`build/stone_storm.aab`)	The file you give to Google Play. Players never see it; Play turns it into the right install for each phone.
APK (`build/stone_storm_debug.apk`)	An install file you can send to a friend directly. Ours is a *debug* APK: test ads, no purchases.
Release build	Real ads, real purchases. Only goes to Google Play.
Debug build	Google's test ads, no purchases, made for testing. Never goes to Play.
Version code	A whole number (7, 8, 9...). Must go up by at least 1 for every upload. Players never see it.

Word	Meaning
Version name	What players see, like `1.0.0`. You choose it.
Upload key (`~/castle_upload.keystore`)	Your digital signature. Google only accepts files signed with it.
App signing key	Google's own key. Google re-signs the game with it before sending it to players (Play App Signing). You never handle it.
Track	A group of players a version goes to: internal, closed, open, production.
Rollout %	On production you can send an update to only part of the players first (for example 20%).
License tester	A Google account that can "buy" things without paying (test card).
AdMob test device	A phone that only gets test ads, even in the real game. Tapping them is safe.
Service account	A robot Google account that lets `tools/release.py` upload for you.
Terminal	The Mac app where you type commands. Open it, then type `cd "/Users/sumit/Documents/Castle Smashers 2"` first, every time.

---

2. Your secret files (back them up today)

These files are **not** on GitHub on purpose. If you lose them you lose control of updates.

File	Where	What it is	If you lose it
`castle_upload.keystore`	`~/castle_upload.keystore`	Upload key	Updates stop until Google resets it (part 15, takes days)
Keystore password	in your head / password manager, and inside `export_credentials.cfg`	Unlocks the key	Same as above
`export_credentials.cfg`	`.godot/export_credentials.cfg` in the project	Godot's copy of the key path + password	Re-enter in Godot (part 14)
`ad_ids.cfg`	project folder	Your real ad unit IDs	Copy them again from AdMob → Apps → Ad units
`google-services.json`	project folder **and** `android/build/`	Firebase settings	Download again: Firebase → Project settings → Your apps
`play-service-account.json`	`~/.castle_secrets/`	Upload robot key (part 3)	Make a new key in Google Cloud

Back them up now and after any change:

```
cd "/Users/sumit/Documents/Castle Smashers 2"
```

```
python3 tools/release.py backup-secrets --to "/Volumes/YOUR_USB_NAME"
```

This makes `castle_secrets_DATE.zip`. Put one copy on a USB stick and one in your Google Drive. It contains the keystore password, so never share it or post it anywhere.

Never commit these files to git, never paste the password into a chat, an email or an AI tool.

---

3. One-time setup: let the tool upload to Google Play

Without this you can still upload by hand (part 5, step 6B). With it, one command uploads. It takes about 20 minutes, plus up to a day before Google switches it on.

3.1 Turn on the Play API

1. Open https://console.cloud.google.com and sign in with the same Google account as Play Console.
2. At the top, click the project picker and choose **castle-smashers-a82d6** (your Firebase project).
3. Left menu → **APIs & Services** → **Library**. Search **Google Play Android Developer API** → click it → **Enable**.

3.2 Create the robot account

1. Left menu → **IAM & Admin** → **Service accounts** → **+ Create service account**.
2. Name: `play-uploader`. Click **Create and continue**. Skip the role (click **Continue**), then **Done**.
3. Copy its email. It looks like `play-uploader@castle-smashers-a82d6.iam.gserviceaccount.com`.

3.3 Download its key

1. Click the new service account → tab **Keys** → **Add key** → **Create new key** → **JSON** → **Create**. A `.json` file downloads.
2. In Terminal (change the file name to the one that downloaded):

```
mkdir -p ~/.castle_secrets
mv ~/Downloads/castle-smashers-a82d6-XXXXXXXX.json ~/.castle_secrets/play-service-account.json
chmod 600 ~/.castle_secrets/play-service-account.json
```

3.4 Give it permission in Play Console

1. Play Console (the main page with all your apps) → **Users and permissions** → **Invite new users**.
2. Email address: paste the service account email from 3.2.
3. **App permissions** → **Add app** → Stone Storm → tick:
4. View app information (read only)
5. Release apps to testing tracks
6. Release to production, exclude devices, and use Play App Signing
7. Manage testing tracks and edit tester lists
8. **Invite user** → **Send invitation**. (A service account accepts by itself.)

3.5 Test it

```
python3 tools/release.py status
```

You should see your tracks (internal shows code 4). If it says "could not log in" or "refused", wait a few hours (Google can take up to 24 h) and try again. Then back up the secrets again (part 2).

4. Test mode and real mode: ads, purchases, testers

4.1 How the game chooses test or real ads (automatic)

- **Debug build** (`release.py apk`) always uses Google's test ads. Nothing to change.
- **Release build** (`release.py aab`) uses your real ad units from `ad_ids.cfg`.
- The AdMob **App ID** (`ca-app-pub-1292430265619610~9195063737`) is in `project.godot` and is the same in both builds; that is normal.
- `release.py check` refuses to build if a real ID is missing or a test ID slipped in. You never swap IDs by hand.

Build	Ads	Purchases	Goes to
Debug APK	Google test ads	Don't work	Friends (sideload)
Release AAB, on a registered test phone	Test ads	Test card (license testers)	Play test tracks
Release AAB, everyone else	Real ads, real money	Real money	Play

4.2 Keep real ads safe (AdMob rules)

- **Never tap real ads yourself** and never ask friends or family to tap them. AdMob can close your account for this ("invalid traffic"), and the money is taken back.
- Add every phone you test on as an **AdMob test device**: AdMob → **Settings** → **Test devices** → **Add test device** → platform Android → the phone's advertising ID (phone **Settings** → **Google** → **Ads**; on some phones **Settings** → **Privacy** → **Ads**). Then the real game shows "Test Ad" on that phone.
- If you see an ad without the "Test Ad" label on your phone: don't tap it, close it, and check the test device list.

4.3 Keep purchases safe (Play rules)

- **License testers** never pay. Their purchase sheet says **"Test card, always approves"**. Play Console (main page, not inside the app) → **Settings** → **License testing** → add the Gmail addresses → **Save**. Wait 15 minutes.
- If the sheet shows a real card, press **Back**: that account is not a license tester.
- Test purchases appear in **Order management** with a "Test" label and are never charged.
- Purchases only work in the copy installed **from Play**, never in the debug APK.

4.4 The four kinds of testers

Who	Where to add them	Gets
Internal testers (up to 100)	Test and release → Testing → **Internal testing** → Testers	New builds within minutes, no review
Closed testers	Testing → **Closed testing** → your track → Testers	Builds after a short review; needed for the 12-tester rule
License testers	Settings → License testing	Free test purchases (add your testers here too)
AdMob test devices	AdMob → Settings → Test devices	Test ads in the real game

A tester opens the **opt-in link** (shown on the Testers tab), taps **Become a tester**, then installs from the Play Store link.

---

5. Making a new version (the everyday steps)

Do this every time something in the game changed.

Step 1. Open Terminal in the project

```
cd "/Users/sumit/Documents/Castle Smashers 2"
git pull
```

Step 2. Check everything is ready

```
python3 tools/release.py check
```

Every line must say `[ OK ]` (a `[WARN]` is fine). For any `[FAIL]` the line tells you what to do.

Step 3. Set the new version

```
python3 tools/release.py version --name 1.0.1
```

This adds 1 to the version code and sets the name everywhere. Name rule: small fix → last number +1 (1.0.0 → 1.0.1); new levels or features → middle number +1 (1.0.1 → 1.1.0); huge change → first number +1. Leave out `--name` to just add 1 to the last number.

Step 4. Run the tests (5 to 15 minutes)

```
python3 tools/release.py test
```

Must end with `Tests: OK`. If not, don't release; fix the problem first (part 17).

Step 5. Build

```
python3 tools/release.py aab
python3 tools/release.py apk
```

`aab` builds and then checks the bundle: your signature, version, real ad IDs, billing, notifications, backup, landscape payment screen. It must end with `Verify release bundle: OK`. `apk` is only for friends; skip it if you don't need it.

Step 6A. Upload with the tool (after part 3)

```
python3 tools/release.py upload --track internal --notes "New levels and bug fixes."
```

`--notes` is the "What's new" text players see (max 500 characters).

Step 6B. Or upload by hand 1. Play Console → Stone Storm → **Test and release** → **Testing** → **Internal testing** → **Create new release**. 2. **Upload** → pick `build/stone_storm.aab` (in Finder: the project folder → `build`). 3. Release name: leave as is. **Release notes**: write the "What's new" text between the `<en-US>` lines. 4. **Next** → read any warnings (part 15) → **Save and publish**. 5. Add a row to `docs/RELEASE_HISTORY.md` with the date, code and name.

Step 7. Test it on your phone from Play Play Store → Stone Storm → **Update** (if it doesn't show, close the Play Store and open it again, or wait 10 to 30 minutes). Play a few levels, open the shop, watch one rewarded ad.

Step 8. Save the work to GitHub

```
git add -A
git commit -m "Release 1.0.1 (code 8)"
git tag v1.0.1-8
git push origin main --tags
```

Shortcut: steps 2 to 6A and the git tag in one command:

```
python3 tools/release.py release --name 1.0.1 --track internal --notes "New levels and bug fixes." --push
```

---

6. Play tracks: internal, closed, open, production

Track	Who gets it	Review	Use it for
Internal	Up to 100 testers you list	None, minutes	Every new build first
Closed (alpha)	Testers you list, any number	Short review (hours)	The 12-testers/14-days rule; bigger tests
Open (beta)	Anyone who joins from the Play page	Review	Optional public beta
Production	Everyone	Review (hours to a few days; the first one can take up to a week)	The real release

Moving a tested build from one track to another (no new upload needed):

```
python3 tools/release.py promote --from internal --to closed
python3 tools/release.py promote --from closed --to production --rollout 0.2 --confirm
```

By hand: open the target track → **Create new release** → **Add from library** → pick the version.

---

7. Before the first public release (checklist)

7.1 Closed test: 12 testers for 14 days (required for new personal accounts)

Google requires accounts like yours to run a closed test with **at least 12 testers who stay opted in for 14 days in a row** before production is unlocked. Play Console's **Dashboard** shows the counter. 1. Testing → **Closed testing** → **Create track** (or use **Alpha**) → **Testers** tab → **Create email list** → add at least 12 Gmail addresses (15 is safer, in case someone leaves) → **Save**. 2. Add the same emails to **License testing** (part 4.3) so their test purchases are free. 3. Promote the current build: `python3 tools/release.py promote --from internal --to closed` (or by hand, part 6). 4. Send the testers the **opt-in link** from the Testers tab. Each one taps **Become a tester** and installs the game. They should keep it installed and open it every few days. 5. Ship at least one update during the 14 days (Google likes to see the game improving from feedback). 6. After 14 days: Dashboard → **Apply for production**. Answer honestly: how you found testers, what feedback you got, what you changed. Google replies in about a week.

7.2 App content forms (Play Console → Policy and programs → App content)

Every item must show a green tick.

Form	Answer
Privacy policy	`https://stonestorm.pages.dev/privacy/`
App access	All functionality is available without special access
Ads	**Yes, my app contains ads**
Content rating	Fill the questionnaire: category **Game**; cartoon/fantasy violence **yes** (smashing castles, cartoon archers knocked out); no blood, no real violence; **in-app purchases yes**; users can't talk to each other; no sharing of location. Submit and apply the rating.
Target audience and content	Ages **13 and over** only (13–15, 16–17, 18+). We did **not** design it for children, so don't tick under-13 groups (that would trigger the strict Families rules and need different ads). If Google says the store listing could appeal to children, answer honestly and explain the 13+ choice.
Data safety	Use the answers in `docs/FIREBASE_SETUP.md` (section "Data safety"). Data is encrypted in transit: **yes**. Deletion requests: **yes**, by email to intercoreapp@gmail.com. The Android backup of the save file is Google's own backup, not data you collect.
Advertising ID	**Yes**, used for **Advertising or marketing** and **Analytics**
Government apps	No
Financial features	None
Health	None
News app	No

7.3 Store listing

Everything is ready in `store_listing/listing.md` (text to copy) and `store_listing/` (images). Grow users → Store presence → **Main store listing** → paste and upload → **Save**. Then **Store settings**: category **Games** → **Casual**, email and website as in `listing.md`.

7.4 Countries and price

- Production → **Countries / regions** → **Add countries / regions** → choose all (or the ones you want).
- The app is **Free**. A free app can never be made paid later.
- In-app product prices: Monetize with Play → Products → In-app products → each product → **Purchase options** → **Edit prices**. Play converts to each country; rounding is fine.

7.5 AdMob after the game is live (very important for earnings)

1. When the game is public on Play: AdMob → **Apps** → Stone Storm → **App settings** → **Add app store details** (or **Link to app store**) → search "Stone Storm" → pick it. Until you do this, AdMob serves few or no real ads.
2. AdMob checks the app ("app review"). Wait until the app shows **Ready**.
3. **app-ads.txt**: AdMob → Apps → **app-ads.txt** tab should show "Found" within a few days. It reads `https://stonestorm.pages.dev/app-ads.txt`, so the **website field in the store listing must be exactly**

`https://stonestorm.pages.dev`.
4. Check **Privacy & messaging**: GDPR and US state messages say **Published**.

7.6 Last checks

- [ ] Latest build passed `release.py check`, `test` and `aab`
- [ ] Played it from the closed track on a phone: purchases (test card), rewarded ad, interstitial
- [ ] Pre-launch report (Test and release → **Pre-launch report**): no crashes
- [ ] All App content forms green, store listing saved, countries chosen
- [ ] Secrets backed up (part 2)

---

8. Going public: production and staged rollout

1. Production access approved (part 7.1).
2. Promote the closed build to production for **20% of players** first:

```
python3 tools/release.py promote --from closed --to production --rollout 0.2 --confirm
```

(By hand: Production → Create new release → Add from library → set rollout % → Save → Review → Start rollout.) 3. Wait for review (Publishing overview shows the status). The first review can take up to 7 days. 4. When it's live, watch for 1 to 2 days: Android vitals (crashes), Firebase Crashlytics, reviews. 5. All good → more players:

```
python3 tools/release.py rollout --track production --fraction 0.5 --confirm
python3 tools/release.py rollout --track production --fraction 1.0 --confirm
```

1. Something badly wrong → stop the rollout (players who already have it keep it):

```
python3 tools/release.py halt --track production --confirm
```

Then fix the problem and ship a new version (part 10.4).

Tip: **Publishing overview** → **Managed publishing** (switch on) lets you choose the moment an approved update goes live, instead of right after review.

---

9. Changing things later: what needs a new version

What you want to change	Where	New version needed?
Store text, screenshots, icon on the Play page	Play Console → Main store listing	No
Product price	Play Console → Products → the product → Purchase options	No
Product title / description on the Play sheet	Play Console → Products	No
Testers, countries, rollout %	Play Console	No

What you want to change	Where	New version needed?
Push message to players	Firebase → Messaging → New campaign → topic `all_players`	No
Ad settings (block ad categories, frequency caps)	AdMob → Blocking controls / app settings	No
Website, privacy policy	edit `web/`, then `python3 tools/release.py web`	No (but tell Play if the policy changes a lot)
How often interstitial ads show	`data/balance.json` → `"ads"` (`interstitial_every_matches`, `interstitial_min_level`, `interstitial_min_gap_s`)	**Yes**
Coins/gems in a crate, upgrade costs, energy, rewards	`data/balance.json`	**Yes**
Reminder texts and timing	`data/notifications.json`	**Yes**
Levels	`tools/campaign.py`, then `python3 tools/gen_levels.py`	**Yes**
A new ad unit ID	`ad_ids.cfg`	**Yes**
A new in-app product	Play Console **and** `data/balance.json` (see below)	**Yes**
Bug fixes, art, sounds, code	the project	**Yes**

Adding a new in-app product 1. Play Console → Monetize with Play → Products → In-app products → **Create product**. Product ID in small letters with `_` (for example `gem_chest`). It can **never** be changed or reused. Purchase option ID `buy`, type Buy, set price → **Activate**. 2. In `data/balance.json`, under `"shop"`, add the ID to `"order"` and an entry in `"products"` copied from a similar one (`coin_crate` for things that can be bought again, `addon_crown` for one-time items). 3. New version (part 5). Test the purchase with a license tester before production.

Never delete or rename an existing product; people who bought it need it to keep working. To stop selling one, set it to **Inactive** in Play Console and remove it from `"order"`.

Store listing pictures (new screenshots)

```
G=/Users/sumit/Downloads/Godot.app/Contents/MacOS/Godot
$G --path . --resolution 1920x1080 -- --level=LEVEL_NUMBER --no-plan --no-intro --no-popups --sharp-bg --autoshot=38,0.82,0.9 --screenshot=store_listing/screenshots/NEW.png --shot-delay=5.4
```

`--shot-delay` is the second the picture is taken (try 5 to 6 to catch the hero in the air).

---

10. How players get updates

10.1 What players experience

- Most phones update apps automatically (usually on Wi-Fi, within 1 to 3 days). Others tap **Update** in the Play Store.
- Their progress, coins, Remove Ads and hats stay: same package name, same signature.

- On first start after an update the studio intro plays again, and 3 hours after they leave the game they get one "New in the kingdom" reminder.

10.2 How long old versions keep working

Forever. Google never forces an update. A player on an old version keeps playing it until they update. Purchases keep working on old versions too.

This means: never change something on a server or in Play Console that would break old versions (for example, don't delete a product ID).

The game also offers updates itself (from version 0.5.3). Once per session, on the home screen, it asks Play whether a newer version exists: - **Normal update** (priority 0 to 3, the default): Play shows a small "Update available" box. If the player says yes it downloads in the background while they keep playing, and installs the next time they leave the game. If they say no, the game asks again after 24 hours. - **Critical update** (priority 4 or 5): Play shows a full-screen "Update now" screen and the game can't be played until it's updated. Use this only for serious bugs or broken purchases.

You choose the priority when you upload (it can't be changed later for that version):

```
python3 tools/release.py upload --track production --priority 0 --notes "New levels." --confirm
# normal
python3 tools/release.py upload --track production --priority 5 --notes "Fixes a crash." --confirm
# forces the update
```

Uploading by hand in Play Console always gives priority 0 (Play Console has no field for it). The update box only appears for copies installed from Play, and only when the newer version has reached that player (for example, inside the rollout percentage).

10.3 Telling players about an update

- Write a good **"What's new"** text (`--notes`).
- Optional: Firebase → **Messaging** → **New campaign** → **Notifications** → title + text → target: **Topic** `all_players` → send. Only players with notifications on receive it.

10.4 Fixing a bad update (no "undo")

Play can't go back to an older version. To undo, ship a **new** version (higher code) with the fix or with the old code: 1. `python3 tools/release.py halt --track production --confirm` (if it's still rolling out). 2. Fix the bug (or `git revert` the bad change), then part 5 with a new version. 3. Upload to internal, test, promote to production.

10.5 Every year: target Android version

Each year (deadline around **31 August**) Google requires updates to target a recent Android version. The game targets Android 16 (API 36) now, which is enough until about August 2027. Play Console shows a warning months before. The fix is usually updating Godot and its Android build template, then rebuilding. A developer or a strong AI should do this one.

10.6 Billing and ad library updates

Google retires old versions of its billing library about two years after release (we use 9.1.0) and AdMob does the same with its SDK. Play Console and AdMob email you. Old installs keep working; only new uploads are blocked. Updating means updating the plugin in `addons/` (developer job).

---

11. After release: what to check and when

Every day for the first week, then twice a week - Play Console → **Quality** → **Android vitals** → Crashes and ANRs. Google's "bad behaviour" limits: about **1.09%** of users with crashes and **0.47%** with freezes (ANRs). Above that, Play shows the game less. - Firebase → **Crashlytics**: new crashes, which level, which phone. - **Ratings and reviews**: reply politely, especially to 1 and 2 star reviews. The game asks for ratings with Google's own review card: after a 3-star win from level 8, at most 3 times, 30 days apart (settings in `data/balance.json` → `"play"`). Google decides whether the card actually appears and limits how often. Never offer rewards for ratings: that breaks Play policy.

Every week - AdMob → **Home**: earnings, match rate. A sudden drop to 0 → check **Policy center**. - Play Console → **Inbox** and your email: policy messages have deadlines, act quickly. - Firebase → **Analytics**: where players stop (level_failed events), how many come back.

Every month - Payments (part 12). - `python3 tools/release.py check` still passes on your Mac. - Back up secrets if anything changed (part 2).

Every year - Target Android version (part 10.5). - Renew anything with an expiry (your upload key is valid until 2054, nothing else expires). - Keep logging in to Play Console and AdMob: Google can close accounts that stay inactive for a long time, and new rules arrive by email (for example Google's developer verification for apps installed outside Play, rolling out from 2026: your Play account is verified; if friends one day can't install the debug APK, add them as closed testers instead).

---

12. Money: Play earnings, AdMob earnings, refunds, taxes

Google Play (in-app purchases) - Google keeps a service fee (15% of the first US$1 million a year for most developers) and pays the rest monthly to the bank in your payments profile (around the 15th, for the month before, once over the minimum). - See it: Play Console → **Monetize with Play** → **Financial reports**, and https://pay.google.com → Payments profile.

AdMob (ads) is a separate payment - AdMob → **Payments**: fill tax information and a payment method (bank account) there too. - When you reach about US$10, Google posts a **PIN letter** to your address. Enter the PIN in AdMob → Payments within 4 months, or payments stop. - Payment comes around the 21st each month once you pass the payment threshold shown on the Payments page (about US$100).

Refunds - Players can ask Google for a refund themselves (usually within 48 hours of buying). - You can refund: Play Console → **Order management** → find the order → **Refund**. - After a refund of Remove Ads or a hat, Play removes it at the next restore. Coins from a refunded crate stay in the player's game (there is no server to take them back).

Taxes (India) Play and AdMob income are business income. Ask a CA (chartered accountant) about GST and income tax, and keep the monthly reports from Play and AdMob.

---

13. Your phone, friends' phones and saved progress

- **Your phone** has the game from Play (internal testing). Always update it from the Play Store. Don't uninstall it to try the debug APK.
- **Debug APK and Play version can't be installed together**: they have different signatures, so Android refuses to install one over the other. Switching means uninstalling.
- **Friends who aren't testers**: send them `build/stone_storm_debug.apk` (WhatsApp, Drive). They allow "Install unknown apps" for the app they open it with. Test ads only, no purchases.
- **Friends who should test properly**: add them as closed testers (part 4.4) instead.

What survives an uninstall and reinstall from Play (from version 0.5.2) | Thing | Comes back? | |---|---| | Levels, stars, achievements, coins, gems, upgrades, name | **Yes, if** Google backup is on (phone Settings → Google → Backup → Backup by Google One **on**) and Android has made a backup (it does this about once a day while charging on Wi-Fi). Otherwise no. | | Remove Ads, Rocky hats | **Yes**, always: open the Shop (Play restores them) | | Unused crates | Already turned into coins: they come back only with the backup |

Force a backup before you uninstall (phone on USB):

```
~/Library/Android/sdk/platform-tools/adb shell bmgr backupnow com.castlesmashers.game
```

The debug APK has its own separate save; it starts from zero and its progress isn't carried over to the Play version.

See what's on the phone:

```
python3 tools/release.py phone
python3 tools/release.py logs      # game log, Ctrl+C to stop
```

---

14. Setting up a new Mac

1. Install: **Godot 4.7** (from godotengine.org; put it in Downloads or set `GODOT=/path/to/Godot`), **Android Studio** (it installs the Android SDK; then SDK Manager → install Android SDK 36, Build-Tools, Platform-Tools), **Java 17** (Android Studio includes one), **Python 3**, **Node.js** (for the website), **Git**, **Google Chrome**.
2. Get the project:

```
cd ~/Documents
git clone https://github.com/sumit-dev06/Castle-Smashers.git "Castle Smashers 2"
cd "Castle Smashers 2"
pip3 install google-api-python-client google-auth pillow
```

1. Restore secrets from your backup zip: `castle_upload.keystore` → `~/`, `ad_ids.cfg` and `google-services.json` → the project folder, `play-service-account.json` → `~/.castle_secrets/`.
2. Open the project in Godot → **Project** → **Install Android Build Template**, then:

```
cp google-services.json android/build/
python3 tools/patch_android_template.py
```

1. Godot → **Project** → **Export** → **Android** → **Keystore** → **Release**: pick `~/castle_upload.keystore`, user `upload`, your password. (This creates `.godot/export_credentials.cfg`.) Also set **Editor** → **Editor Settings** → **Export** → **Android** → **Java SDK and Android SDK paths** if empty.
2. Website login once: `npx wrangler login`.
3. `python3 tools/release.py check` → everything `[ OK ]`.

---

15. Troubleshooting

Problem	What to do
`check`: keystore password wrong	Godot → Project → Export → Android → Keystore Release password

Problem	What to do
`check`: project.godot version doesn't match	`python3 tools/release.py version --keep-code --name THE_NAME`
`check`: version code already used	`python3 tools/release.py version`
`check`: export_format is not 0	A build was interrupted. Open `export_presets.cfg`, set `gradle_build/export_format=0`
`test` fails	Don't release. Give the failing lines (in `build/tmp/tests.log`) to a developer/AI (part 17)
Build fails	Look in `build/tmp/export_release.log`. Common: Android build template missing (part 14 step 4)
Play: "Version code X has already been used"	`release.py version`, then `aab` again
Play: "signed with the wrong key"	The AAB wasn't signed with `~/castle_upload.keystore`. Run `release.py verify`
Lost the upload key or password	Play Console → **Test and release** → **App integrity** → **App signing** → **Request upload key reset**. Make a new key: `keytool -genkeypair -v -keystore ~/castle_upload_new.keystore -alias upload -keyalg RSA -keysize 2048 -validity 10000`, export the certificate: `keytool -export -rfc -keystore ~/castle_upload_new.keystore -alias upload -file upload_certificate.pem`, upload the `.pem`. Google takes a few days.
Shop says "Loading..."	Installed by USB instead of Play, products not Active, or a non-tester account. Wait an hour after creating products.
Ads don't show	New ad units take up to an hour; real ads need the store link (part 7.5); Remove Ads owned
Ad without "Test Ad" on your phone	Don't tap. Add the phone as an AdMob test device
"Changes not sent for review" after upload	Play Console → **Publishing overview** → **Send changes for review**
Upload saved as **draft**	Open the track in Play Console → **Edit release** → **Review** → **Start rollout**
`status`/`upload`: could not log in / refused	Redo part 3.4; wait up to 24 h after inviting
Game update not showing on the phone	Close the Play Store, open again; check the phone's account is a tester; wait 30 min

Problem	What to do
Policy email from Google	Read the deadline. Most issues are fixed in App content forms or the store listing. If it needs code, get a developer.

---

16. Command cheat sheet

Always start with `cd "/Users/sumit/Documents/Castle Smashers 2"`.

Command	Does
`python3 tools/release.py check`	Is everything ready?
`python3 tools/release.py version --name 1.0.1`	New version (+1 code)
`python3 tools/release.py test`	All automatic tests
`python3 tools/release.py aab`	Build + check the Play bundle
`python3 tools/release.py apk`	Debug APK for friends
`python3 tools/release.py verify`	Re-check the bundle
`python3 tools/release.py upload --track internal --notes "TEXT"`	Send to Play
`python3 tools/release.py upload --track internal --dry-run`	Check without sending
`... upload ... --priority 5`	Critical fix: players must update before playing (part 10.2)
`python3 tools/release.py status`	What's on each track
`python3 tools/release.py promote --from internal --to closed`	Move a build to another track
`python3 tools/release.py promote --from closed --to production --rollout 0.2 --confirm`	Go public to 20%
`python3 tools/release.py rollout --track production --fraction 1.0 --confirm`	Everyone gets it
`python3 tools/release.py halt --track production --confirm`	Stop a rollout
`python3 tools/release.py release --name 1.0.1 --track internal --notes "TEXT" --push`	Everything in one go
`python3 tools/release.py phone` / `logs`	USB phone info / game log
`python3 tools/release.py backup-secrets --to "/Volumes/USB"`	Back up keys
`python3 tools/release.py web`	Publish the website

Tracks: `internal`, `closed`, `open`, `production`. Production always needs `--confirm`. Every upload/promote/rollout is written to `docs/RELEASE_HISTORY.md`.

---

17. Working with an AI helper

Give the AI these files first: `AGENTS.md`, `docs/RELEASE_GUIDE.md` (this file) and, for code changes, `docs/ARCHITECTURE.md`. Then ask in plain words. Example requests:

- *Release:* "Read docs/RELEASE_GUIDE.md. Make a new version 1.0.2 with notes 'Bug fixes'. Follow part 5 exactly, stop and show me any [FAIL] line."

- *Balance:* "In data/balance.json, change interstitial_every_matches from 4 to 5. Run `python3 tools/release.py test`. Don't change anything else."
- *Test failure:* "`python3 tools/release.py test` failed. Read build/tmp/tests.log, find the cause, fix it with the smallest change, run the tests again."
- *Policy email:* "Here is Google's email: [paste]. Tell me which Play Console form or file to change, step by step."

Rules for the AI (copy them into your request if it's a simple model): 1. Never commit or print `ad_ids.cfg`, `google-services.json`, keystores, passwords or `~/.castle_secrets`. 2. Never uninstall the game from the phone or install the debug APK over the Play version. 3. Never use `--confirm` for production unless the owner said so in this conversation. 4. Version codes only go up. Use `release.py version`, never edit them by hand. 5. Product IDs and the package name `com.castlesmashers.game` never change. 6. After any change: `python3 tools/release.py test` must pass before building. 7. Tunable numbers go in `data/*.json`, not in code.